

Data Sorting in Flash Memory

CHIN-HSIEN WU and KUO-YI HUANG, National Taiwan University of Science and Technology

Because flash memory now provides an economical solution for various portable devices and embedded systems, an NAND flash-based storage system has replaced the hard disk drive in many applications. Recently, the implementation of database systems using an NAND flash-based storage system has become an important research topic. In particular, the external sorting is an important operation in database systems. With the very distinctive characteristics of flash memory, the typical external sorting system that adopts a clustered sorting process can result in performance degradation and reduce the reliability of flash memory. In this article, we will propose an unclustered sorting method that considers the unique characteristics of flash memory, and we then propose a decision rule to exploit the advantages of both clustered and unclustered sorting. The decision rule can separate records according to their record length, sort them appropriately by the clustered and unclustered sorting, and merge the sorted results. The experimental results show that the proposed method can improve performance in an NAND flash-based storage system (i.e., solid-state drive).

Categories and Subject Descriptors: E.5 [Files]: Sorting/Searching; D.4.2 [Operating Systems]: Storage Management; Secondary Storage; B.3.2 [Memory Structures]: Mass Storage

General Terms: Algorithm, Design, Performance

Additional Key Words and Phrases: Flash memory, memory structures, sorting/searching, storage management

ACM Reference Format:

Chin-Hsien Wu and Kuo-Yi Huang. 2015. Data sorting in flash memory. *ACM Trans. Storage* 11, 2, Article 7 (March 2015), 25 pages.

DOI: <http://dx.doi.org/10.1145/2665067>

1. INTRODUCTION

Flash memory has become a popular storage system alternative because of its advantages of nonvolatility, shock resistance, and low-power consumption. In recent years, flash memory devices have advanced along with a new wave of consumer electronics and embedded systems. Flash memory can be considered as an alternative to hard disk drives in many applications. For example, an NAND flash-based Solid-State Drive (SSD) has become popular and replaced hard disk drives. Many manufacturers (e.g., SanDisk, STEC, Samsung, Mtron, Intel, and PNY) have released extremely large-capacity NAND flash-based SSDs (e.g., 512GB SSD) on the market. Currently, implementation of the external sorting, which is an important operation in disk-based storage systems, must be considered with regard to NAND flash-based storage systems.

This work was supported in part by a research grant from the Ministry of Science and Technology under Grant MOST 103-2221-E-011-065-MY2.

Authors' addresses: C.-H. Wu and K.-Y. Huang, #43, Sec. 4, Keelung Rd., Taipei 106, Taiwan; emails: {chwu, M9902122}@mail.ntust.edu.tw.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701 USA, fax +1 (212) 869-0481, or permissions@acm.org.

© 2015 ACM 1553-3077/2015/03-ART7 \$15.00

DOI: <http://dx.doi.org/10.1145/2665067>

Four critical issues have a significant impact on the efficiency of external sorting in flash memory: (1) write once with bulk erase, (2) the endurance issue, (3) asymmetrical operations, and (4) no mechanical latency. Flash memory cannot be overwritten (updated) unless it is erased. As a result, out-of-date (or invalid) versions and the latest copy of data might coexist simultaneously on flash memory. Furthermore, an erasable unit of flash memory is relatively large so that valid data copying might be involved in the erasure. Frequent erasing of some particular locations of flash memory could quickly deteriorate the overall lifetime of flash memory, because each erasable unit has a limited number of erasure cycles available. In particular, reads and writes on flash memory are different from those on hard disk drives. Reads on flash memory have a faster access speed and less energy consumption than writes on flash memory. Therefore, reads and writes on flash memory are asymmetrical operations. Because flash memory has no disk head and disk platter, no seek or rotational latency will be needed. Therefore, random access and sequential access to flash memory will not cause mechanical latency problems.

The typical external sorting adopts a clustered sorting. Because the clustered sorting is used to sort records in a contiguous space, the sorted records in disk-based storage systems have better access performance. With the very distinctive characteristics of flash memory, the clustered sorting might result in performance degradation and reduce the reliability of flash memory. Therefore, we will adopt an unclustered sorting method that considers flash memory characteristics in this article. The unclustered sorting is a type of tag sorting and creates small tags that point to records. Instead of directly sorting the records, the tags are sorted in order to decrease the number of page reads and writes during the process of merging records. After these tags are sorted, we can rearrange the original records by the sorted tags. We will show the pros and cons for both clustered and unclustered sorting in NAND flash-based storage systems. A decision rule is then proposed to exploit the advantages of both clustered and unclustered sorting. The major contributions of this article are as follows:

- We propose a record rearrangement and replacement method for the unclustered sorting. Record rearrangement is proposed to rearrange the records by scanning the sorted tags. Record replacement is proposed to avoid unnecessary page reads during record rearrangement.
- We propose a decision rule to exploit the advantages of both clustered and unclustered sorting. The decision rule can separate records according to their record length, sort them appropriately by the clustered and unclustered sorting, and merge the sorted results.
- The concept of reused data is proposed to decrease the number of page writes by detecting whether or not the content of the output buffer is identical to some page in flash memory. The concept can be integrated into the address translation table. That is, if any pages will be reused, its logical addresses will be marked in the address translation table.
- We derive an I/O analysis of the clustered and unclustered sorting for NAND flash-based storage systems. Moreover, we show the I/O comparison of the clustered sorting, the unclustered sorting, MinSort [Cossentine and Lawrence 2013], and FAST [Park and Shim 2009] in terms of number of page reads and writes. Based on the I/O analysis, we are able to gain additional insights into the decision rule.
- We have implemented and performed the proposed method in a real SSD (i.e., an Intel SSD) and a hard disk drive (i.e., a Hitachi HDD). The experimental results show that the proposed method can improve performance when compared to the typical external sorting. We also compare with previous research (FAST and MinSort) and demonstrate that the decision rule can provide an efficient external sorting method for variable-length records.

In this article, Section 2 provides an overview of flash memory characteristics. Section 3 describes the related work and the research motivation. Section 4 introduces both clustered and unclustered sorting. Section 5 compares the clustered and unclustered sorting and then provides a decision rule by exploiting their advantages. Section 6 provides the I/O analysis. Section 7 is the performance evaluation. Section 8 is the conclusion.

2. FLASH MEMORY CHARACTERISTICS

There are two major types of flash memory in the market: NAND flash memory and NOR flash memory. NAND flash memory is mainly used for data storage, while NOR flash memory is often used for code storage. In this article, we focus our discussions on NAND flash memory because it is more suitable to the designs of file/storage systems. An NAND chip consists of many blocks, and each block has a fixed number of pages (e.g., 32 pages or 128 pages) [Samsung Electronics 2011]. A block is the smallest unit for erase operations, while reads and writes are done in pages. A page contains a user area (e.g., 512B or 2KB) and a spare area (e.g., 16B or 64B), where the user area is for the storage of a logical block and the spare area stores ECC and other housekeeping information (i.e., LBA). Because flash memory is a write-once system, we do not overwrite data on each update. Instead, data are written to free space, while the old versions of data are invalidated and considered dead. Such an update strategy is called “out of place” or “write once” updating. In other words, any existing data on flash memory cannot be overwritten (updated) unless the block where it resides is erased. Pages that store live data and dead data are called “valid pages” and “invalid pages,” respectively.

After a number of page writes, free space on flash memory becomes low due to invalid pages. Garbage collection, which consists of a series of reads, writes, and erase operations, will be executed to recycle the invalid pages. Therefore, a flash translation layer is required to execute the garbage collection for flash memory management. Furthermore, because flash memory has the characteristic of an out-of-place update, the physical block addresses (PBAs) of valid pages could change over time. Therefore, a flash translation layer has to map each logical block address (LBA) to its physical block address on flash memory. Usually, a flash translation layer usually has a RAM-resident translation table, where each entry of the table (indexed by LBAs) contains the PBA of the corresponding LBA. Note that LBAs are logical addresses of pages mentioned by the operating system, and each PBA has two parts, the block number and the page number in the block.

3. RELATED WORK AND MOTIVATION

3.1. Related Work

Data sorting in database systems is an important operation for grouping with aggregation, duplicate removal, sort-merge joins, and set operations [Zhang and Larson 1997]. However, because of the size limit of main memory, a large file cannot be sorted in main memory, and the external merge sort in disk-based storage systems has been studied extensively. Many previous research studies [Salzberg et al. 1990; Li et al. 1993; Zhang and Larson 1997; Larson and Graefe 1998; Larson 2003; Graefe 2006; Yiannis and Zobel 2007] have proposed various methods for the external merge sort in disk-based storage systems. When data sorting is required in flash memory, the external sorting design on flash-based storage systems will be different from the implementations and considerations in disk-based storage systems. In particular, some research studies (FAST [Park and Shim 2009] and MinSort [Cossentine and Lawrence 2013]) have proposed various methods for flash-based storage systems. They attempt to decrease

Table I. Comparison with Various Methods

	Flash-Based Algorithm	Memory Buffering	Variable-Length Record	Merge Process	Write Overhead
Clustered Sorting		✓	✓	✓	large
FAST	✓	✓		✓	medium
MinSort	✓		✓		small
Unclustered Sorting	✓	✓	✓		small
Decision Rule	✓	✓	✓	✓	small

the number of write requests by generating longer runs, but this method can cause additional read requests.

FAST divides an input file into several regions and sorts each region in main memory as a run. Then, FAST scans each sorted run and maintains a max heap to record the minimum values during the merge phase of the runs. After each merge of runs, the records that the max heap retains can be written to flash memory as a long run. FAST will continue the merge process until a single run is produced. FAST could decrease many page writes but could cause many page reads. MinSort divides an input file into several regions and keeps a minimum value for each region by a dynamic index array in main memory. MinSort will repeatedly scan a record from a page that contains the minimum value and write the scanned records to flash memory in a page size. MinSort will continue the process until all records have been scanned. MinSort performs well when primary keys exhibit a sequential order but will not perform as well when primary keys are randomly inserted. In this article, we discuss the difference between the clustered and unclustered sorting for flash memory. In particular, this article will provide an external sorting design to exploit the advantages of both sorting methods for NAND flash-based storage systems. The method can be used in Park and Shim [2009] and Cossentine and Lawrence [2013] when both clustered and unclustered sorting are considered.

Table I makes a comparison between various methods¹ (i.e., the clustered sorting, FAST, MinSort, the unclustered sorting, and the decision rule). Based on Table I, FAST, MinSort, the unclustered sorting, and the decision rule are designed for flash memory and can be considered as flash-based algorithms. Memory buffering means that the clustered sorting, FAST, the unclustered sorting, and the decision rule can improve performance when available main memory increases. The clustered sorting, MinSort, the unclustered sorting, and the decision rule can consider variable-length records in their designs. The clustered sorting, FAST, and the decision rule use the merge process in their designs. We also classify the methods according to write overhead (i.e., large, medium, and small). Because the decision rule consists of both clustered and unclustered sorting, the decision rule can inherit their advantages. Overall, the decision rule can provide an efficient external sorting for NAND flash-based storage systems.

3.2. Motivation

Sorting is an important operation in database systems. IBM DB2, Informix, Microsoft SQL Server, Oracle 8, and Sybase ASE all adopt the external merge sort for large files. Because the external merge sort in disk-based storage systems might incur a large amount of data access, the manipulations of the external merge sort in flash memory will also introduce significant overhead. A typical external merge sort consists of two

¹Note that the clustered sorting and the unclustered sorting have been briefly introduced in the conference version [Wu 2012]. New and enough materials have been included in the journal article.

Table II. Performance of a Typical NAND Flash Memory (Samsung K9KAG08U1M)

	Page Read 4KB	Page Write 4KB	Block Erase 256KB
Performance (μ s)	25	200	1,500

phases. The first phase includes reading records from a large file to main memory, sorting them in main memory, and then writing them to flash memory as a run. A sequence of runs will be generated by repeating the process. The second phase involves reading appropriate runs to the main memory and then merging them in main memory. These runs will be merged to a long run. When the number of the total runs is reduced to a single run, the external merge sort ends and the last run is the sorted result. During the generation and merging of runs, considerable intermediate data (i.e., many merged runs) will be generated and written to flash memory. In the experiments, sorting a 2.4GB file could result in about 3.6GB of data being written during a typical external merge sort. When flash memory (i.e., SSDs [Desnoyers 2014; Boboila and Desnoyers 2011]) is used as a storage device, updating (or writing) data in flash memory is a very complicated and expensive operation. If a large amount of data is written, this will result in frequent page writes and block erases to flash memory. In terms of system performance for flash memory, the amount of data written should be reduced as much as possible. This observation was the motivation for producing this article.

4. CLUSTERED SORTING AND UNCLUSTERED SORTING

4.1. Overview

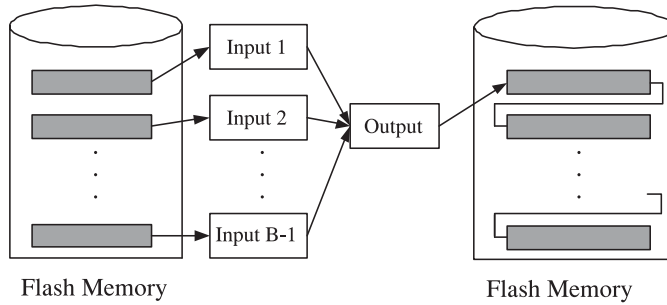
The typical external sorting procedure is a clustered sorting that is designed to sort records in a contiguous space. The clustered sorting can allow better access performance in disk-based storage systems because of its spatial locality (i.e., small mechanical latency). The unclustered sorting is designed to create small tags that point to records and then sort these tags. After these tags have been sorted, we can rearrange the original records by the sorted tags. We have built an NAND-based simulation prototype and conducted experiments to observe both clustered and unclustered sorting. In the experiments, we can observe that the sum of page reads and writes for the unclustered sorting is greater than that of the clustered sorting, as shown in Table III.² Undoubtedly, the clustered sorting is suitable for disk-based storage systems because reads and writes on hard disk drives are symmetrical operations. However, because the unclustered sorting can trade the number of page reads for the number of page writes, the number of page writes and block erases can be less than that of the clustered sorting. For NAND flash-based storage systems, because reads, writes, and erases on flash memory are asymmetrical operations, reads on flash memory have faster access speed than writes and erases on flash memory, as shown in Table II. Therefore, the unclustered sorting may have faster execution time than the clustered sorting, as shown with ♠ in Table III. However, when the record length is 8B, 16B, or 4KB, the clustered sorting could be equivalent or superior to the unclustered sorting. The reason is that the unclustered sorting cannot benefit from small record length (e.g., 8B or 16B) and does not have an opportunity to decrease the number of page reads if a record size (e.g.,

²The detailed simulation setup is in Section 5.1. In the simulation, the page size was 4KB and the block size was 64 pages. The table just shows the comparison under $rs = 0.5$. rs is to control the value distribution of primary keys of records. When rs equals zero, all of the primary keys are randomly generated. If rs equals 1, all of the primary keys are in an ascending order. Consequently, if rs equals 0.5, the values of one-half of the primary keys are in an ascending order, respectively, while the other keys are randomly generated.

Table III. Clustered Sorting Versus Unclustered Sorting

Record Size(B)	Clustered Sorting ($rs = 0.5$) $r+w/e/t$	Unclustered Sorting ($rs = 0.5$) $r+w/e/t$
4,096	20,000,000+10,000,000/156,250/2,734.4	20,036,886+10,019,020/156,547/2739.5
2,048	10,000,000+6,249,567/97,649/1,646.4	12,106,960+5,019,276/78,426/1424.2♣
1,024	5,000,000+3,436,889/53,701/893	7,803,556+2,519,404/39,365/758♣
512	2,500,000+1,797,188/28,081/464.1	5,056,880+1,269,468/19,835/410.1♣
256	1,250,000+917,875/14,341/236.3	3,000,295+644,500/10,070/219♣
128	625,000+463,885/7,248/119.3	1,551,666+332,016/5,187/113♣
64	312,500+233,127/3,642/60	713,451+175,774/2,746/57.1♣
32	156,250+116,872/1,826/30	309,957+97,653/1,525/29.6♣
16	78,124+58530/914/15	136,643+58,592/915/16.5
8	39,062+19,532/305/5.3	78,121+39,062/610/10.7

r = # of page reads (K); w = # of page writes (K); e = # of block erases; t = execution time (seconds).

Fig. 1. Merge of $B - 1$ runs.

4KB) is a multiple of page size. As a result, there are pros and cons for using either the clustered sorting or the unclustered sorting in NAND flash-based storage systems. Therefore, we will propose a decision rule (in Section 5) to exploit the advantages of both clustered and unclustered sorting. The decision rule can be derived by experimental observations (in Section 5.2) and I/O analysis (in Section 6) when variable-length records are considered. That is, the decision rule can separate records according to their record length, sort them appropriately by the clustered and unclustered sorting, and merge the sorted results.

4.2. Clustered Sorting

The typical external sorting [Ramakrishnan and Gehrke 2003] is a type of the clustered sorting that is a merge-sort mechanism. Assume that there are B buffer pages and the size of a record file is N pages. First, during pass 0, read B pages long from the record file into the main memory and sort them in the main memory. After sorting, the sorted content is written to flash memory and is called a run. As a result, $\lceil N/B \rceil$ runs will be generated and each run size is B pages (except the last run could contain fewer pages). In the subsequent passes, $B - 1$ buffer pages are used as the input buffer, where each buffer page has a corresponding run. The last buffer page is used as the output buffer and is written to flash memory while it is full. During each pass, we can merge $B - 1$ runs to a long run and the merge process can be halted until the number of runs becomes one. For example, as shown in Figure 1, $B - 1$ runs in flash memory can be merged to one long run by $B - 1$ input buffer pages and one output buffer page.

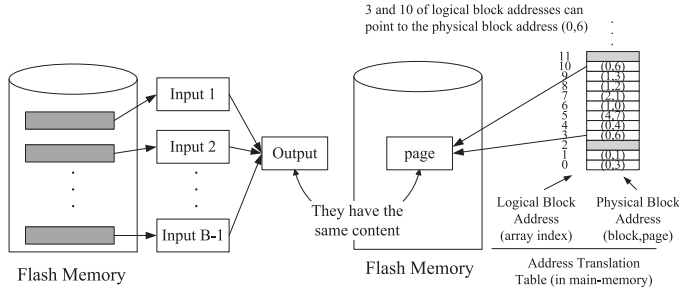


Fig. 2. Concept of reused data.

4.3. Unclustered Sorting

4.3.1. Tag. A tag, which describes each record in a record file, is a tuple (*key*, *pointer*, *len*), where *key* is the primary key of the record, *pointer* points to the record in the file, and *len* is the length of the (variable-length) record. Each record has a corresponding tag, and tags are stored in the tag file. Assume that there are B buffer pages, where $B - 1$ buffer pages are used to buffer the records and the remaining buffer page is the output buffer for the tags. The record file will be scanned to construct tags by B buffer pages. After the tags are created, the unclustered sorting will sort the tags instead of the records. The tag sorting process is similar to that of records in the clustered sorting. If the size of one tag is less than that of one record, many page reads and writes can be reduced during the merging process of runs. Note that although the unclustered sorting is a kind of tag sorting [Knuth 1998], this article will focus on how to exploit the advantages of both clustered and unclustered sorting for NAND-based storage systems. Note that for the sake of simplicity, instead of (*key*, *pointer*, *len*), the type of (*key*, *pointer*) is adopted for the rest of the article.

4.3.2. Reused Data. In the typical external merge sort, the output buffer is written out while it is full. However, we propose detecting whether the content of any page in flash memory is identical to that of the output buffer. If they are identical, the output buffer will not be written and the page can be reused. The objective is to decrease the number of page writes for improving performance. To achieve this objective, we can integrate the concept of reused data into the address translation table of a flash translation layer (FTL). That is, if any pages will be reused, the logical addresses will be marked in the address translation table. When garbage collection will recycle its residing block, the new page address will be updated accordingly in the address translation table. For example, as shown in Figure 2, assume that the content of the output buffer is identical to that of one page whose logical block address is 3 and physical block address is (0,6); then the output buffer will not be written to flash memory and the page can be reused by including the physical block address (0,6) in the address translation table for logical block address 10. Therefore, 3 and 10 of logical block addresses can point to the physical block address (0,6). As a result, the number of page writes and block erases may be reduced. We can adopt the concept of reused data for both clustered and unclustered sorting if the concept can be integrated into the address translation table.

One technical problem is being able to rapidly detect whether the content of the output buffer is identical to any page in flash memory. Because the read/write unit of flash memory is a single page, the size of both the input buffer and the output buffer should be equal to the page size in flash memory. Each run in flash memory consists of pages, and each run is read into the corresponding input buffer one page at a time. As a result, the content of each input buffer is read from a page in flash memory. If the

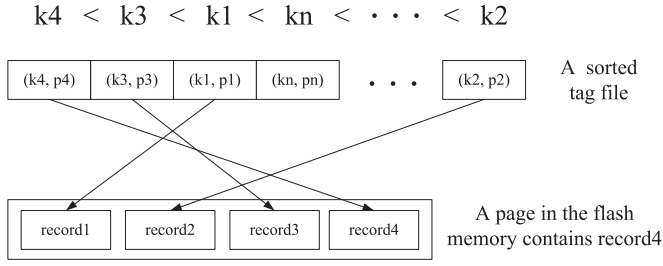


Fig. 3. Case 1.

content of the output buffer and an input buffer are identical, this means a page exists in flash memory that is identical to the content of the output buffer.

Therefore, we can use counters to detect if the output buffer and an input buffer are identical without memory operations (e.g., memcmp). Note that each input buffer has one corresponding counter. Assume that there exists $B-1$ input buffer pages and one output buffer page, and each input/output buffer page can hold R records. Before the merge of the $B-1$ (input buffer) pages, $B-1$ counters will be reset to 0. During the merge of the $B-1$ (input buffer) pages, the corresponding counter of an input buffer is increased by one if the smallest value has been chosen from the input buffer and written to the output buffer. When the output buffer is full, we will check whether one counter is equal to R . If there exists the counter, it means that all values in the corresponding input buffer are always chosen as the smallest value, and there exists a page in flash memory that is identical to the content of the output buffer. After the reused page is marked in the address translation table, the counter will be reset to 0 and the output buffer will be cleaned to continue the next merge of the $B-1$ (input buffer) pages.

4.3.3. Rearrangement and Replacement. In this section, we will rearrange records by scanning the sorted tags that point to the records in the record file. Based on the record order in the sorted tags, the records are read from the original record file and written to a new record file. During this record rearrangement, B buffer pages can be used to buffer records and decrease the number of page reads. However, replacement will be needed when B buffer pages are full. We list three replacement cases as follows:

—Case 1: If the size of a record is smaller than that of a page, a page can contain more than one record. That is, when a record in a page is read, other records in the page will be read and kept in the buffer page. As shown in Figure 3, assume that *record4* will be read by the tag $(k4, p4)$. A page that contains *record4* is read into a buffer page that also contains *record1*, *record2*, and *record3*. To decrease the number of page reads, *record1*, *record2*, and *record3* can be buffered because they might be needed at a later time. Note that when a record in a buffer page is written out, the space occupied by the record can be reused. However, because the number of buffer pages is limited (i.e., B buffer pages), not all records can be kept in the buffer pages. As a result, we propose replacing records that will not be used for the longest period of time. Assume that tags in the sorted tag file are in ascending order. That is, the record that will not be used for the longest period of time must have the largest primary key in the buffer pages. As a result, a priority queue should be maintained for the buffer pages, and the record that has the largest primary key can always be retrieved quickly (i.e., time complexity is $\log(n)$, where n is the number of records in buffer pages).

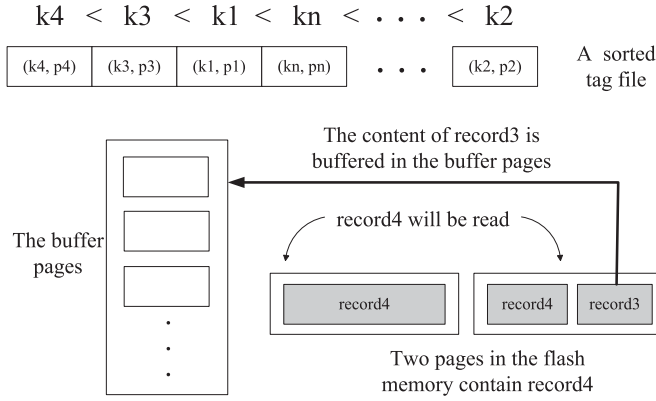


Fig. 4. Case 3.

- Case 2: If the size of a record is a multiple of page size, this means that a record will entirely occupy one or more than one page. As a result, when a page is read, no other records will be in the page and no replacement will be executed.
- Case 3: In addition to Case 1 and Case 2, the size of a record can be larger than the size of a page and not be a multiple of the page size. In this case, the last page could contain the content of other records. As shown in Figure 4, assume that *record4* occupies 1.5 pages, and *record3* occupies half part of the second page. When *record4* is read, *record3* is also read and is kept in the buffer pages. If *record3* needs to be read later, we can skip one page read. Note that the replacement strategy for Case 3 is similar to Case 1.

In addition to Case 2, Case 1 and Case 3 need memory management for buffering the content of records. Compared to the access time of flash memory (i.e., $30\mu s$), the access time of main memory (i.e., $50ns$) is still faster than that of flash memory. As a result, buffering capability is required to improve performance.

5. A DECISION RULE OF THE CLUSTERED AND UNCLUSTERED SORTING

5.1. Simulation Prototype

We have built an NAND-based simulation prototype and conducted experiments to observe the performance of both clustered and unclustered sorting. The simulation prototype was run on a desktop with an Intel Core i7 CPU at 3.07GHz with a 12GB DRAM. The access performance of NAND flash memory was included in Table II. In the simulation prototype, we implemented a hybrid FTL [Lee et al. 2007] that can provide 100GB block-device emulation. In the FTL simulator, the page size was 4KB and the block size was 64 pages. The FTL simulator uses three kinds of blocks, such as data blocks, sequential log blocks, and random log blocks, where data can be stored. Three kinds of merges, such as switch merges, partial merges, and full merges, can be performed to generate free space in the FTL simulator. When data is written to a sequential log block in a sequential and continuous order, its corresponding data block can be erased and the sequential log block will become a new data block. The procedure is called a “switch merge” and no valid pages are read and copied to other space. In fact, when the clustered and unclustered sorting are performed in the simulation prototype, only switch merges (i.e., garbage collection) occur. Note that random log blocks will not be needed and partial/full merges will not occur when the unclustered and clustered sorting are performed in the simulation prototype.

Table IV. File Size and Record Length (Number of Records is 10,000,000)

File Size (MB)	39,062.5	19,531.3	9,765.6	4,882.8	2,441.4	1,220.7	610.4	305.2	152.6	76.3
Record Length (B)	4,096	2,048	1,024	512	256	128	64	32	16	8

According to the definition of Write Amplification Factor (WAF), it means the actual amount of physical information written is a multiple of the logical amount intended to be written for flash memory and SSDs. In fact, the additional amount of physical information written is due to the recycling cost that could contain a series of read, write, and erase operations. In the simulation prototype, new runs can be generated by the clustered and unclustered sorting during the generation and merging of runs. After the new runs are written in a sequential and continuous order, the previous old runs will become invalid pages. Therefore, when the old runs are recycled, no valid pages will be read and copied during the execution of garbage collection (i.e., switch merges). As a result, all WAF values in the simulation prototype are 1.

In the experiments, 40MB of main memory was adopted and 10,000,000 records were placed in a record file to be sorted. Although the size of the main memory was 40MB, it is enough to make a comparison with different file sizes from about 76.3MB to 39,062.5MB, as shown in Table IV. The record length was chosen among the set (4KB, 2KB, 1KB, 512B, 256B, 128B, 64B, 32B, 16B, 8B). The size of a tag was 8B and a ratio rs was used to control the value distribution of primary keys of records. When rs equals zero, all of the primary keys were randomly generated. When rs equals 1, all of the primary keys were in an ascending order. Consequently, when rs equals 0.25, 0.5, and 0.75, the values of one-fourth, one-half, and three-quarters of primary keys were in an ascending order, respectively, while the other keys were randomly generated.

5.1.1. Number of Page Reads. When the value distribution was randomly generated, the number of page reads of the unclustered sorting was more than that of the clustered sorting, as shown in Table V. This was because random primary keys could degrade the buffering ability during the record rearrangement/replacement, and pages could be read frequently from flash memory. When the value distribution was in an ascending order, the number of page reads of the unclustered sorting was decreased and even was close to that of the clustered sorting. This was because the buffering ability during the record rearrangement/replacement had better performance for value distribution in an ascending order. Generally speaking, when rs increased, the number of pages reads of the unclustered sorting decreased during the record rearrangement/replacement. No matter what the value of record length is, the number of page reads of the unclustered sorting was more than that of the clustered sorting due to the record rearrangement/replacement. In particular, when record length was 4KB, the number of page reads of the unclustered sorting was the same under any rs values because the record length was equal to a page size. Therefore, Case 2 was satisfied and no buffering occurred during the record rearrangement/replacement. Therefore, we do not have an opportunity to decrease the number of page reads by the buffer pages in Case 2.

5.1.2. Number of Page Writes. No matter what the value of rs is, the number of page writes of the unclustered sorting in most cases was less than or close to that of the clustered sorting, as shown in Table VI. This was because a small tag could cause fewer page writes during the sorting process. Because we also detected if the output buffer was identical to some page in flash memory, the number of pages writes of the clustered and unclustered sorting was decreased, especially when rs increased. When the record length was decreased, the number of page writes of the unclustered sorting

Table V. Number of Page Reads

Record Length	Sorting Method	$rs = 0$	$rs = 0.25$	$rs = 0.5$	$rs = 0.75$	$rs = 1$
4KB	U	20,036,886	20,036,886	20,036,886	20,036,886	20,036,886
	C	20,000,000	20,000,000	20,000,000	20,000,000	20,000,000
2KB	U	14,420,066	13,254,158	12,106,960	10,988,208	10,038,486
	C	10,000,000	10,000,000	10,000,000	10,000,000	10,000,000
1KB	U	11,094,470	9,427,277	7,803,556	6,251,200	5,039,030
	C	5,000,000	5,000,000	5,000,000	5,000,000	5,000,000
512B	U	8,455,140	6,714,941	5,056,880	3,549,777	2,539,302
	C	2,500,000	2,500,000	2,500,000	2,500,000	2,500,000
256B	U	5,804,532	4,327,789	3,000,295	1,888,570	1,289,438
	C	1,250,000	1,250,000	1,250,000	1,250,000	1,250,000
128B	U	3,300,143	2,344,998	1,551,666	940,273	664,506
	C	625,000	625,000	625,000	625,000	625,000
64B	U	1,566,072	1,088,861	713,451	457,334	352,040
	C	312,500	312,500	312,500	312,500	312,500
32B	U	639,896	422,321	309,957	214,685	195,807
	C	156,250	156,250	156,250	156,250	156,250
16B	U	233,078	175,317	136,643	117,179	117,691
	C	78,124	78,124	78,124	78,124	78,124
8B	U	97,628	92,753	78,121	78,121	78,633
	C	39,062	39,062	39,062	39,062	39,062

Table VI. Number of Page Writes

Record Length	Sorting Method	$rs = 0$	$rs = 0.25$	$rs = 0.5$	$rs = 0.75$	$rs = 1$
4KB	U	10,038,507	10,033,645	10,019,020	10,019,020	10,019,020
	C	10,000,000	10,000,000	10,000,000	10,000,000	10,000,000
2KB	U	5,038,763	5,033,901	5,019,276	5,019,276	5,019,532
	C	7,499,261	6,875,163	6,249,567	5,624,676	5,000,000
1KB	U	2,538,891	2,534,029	2,519,404	2,519,404	2,519,532
	C	4,374,948	3,905,989	3,436,889	2,968,462	2,500,000
512B	U	1,288,955	1,284,093	1,269,468	1,269,468	1,269,532
	C	2,343,304	2,069,901	1,797,188	1,523,465	1,250,000
256B	U	663,987	659,125	644,500	644,500	644,532
	C	1,210,482	1,064,364	917,875	771,568	625,000
128B	U	351,503	346,641	332,016	332,016	332,032
	C	615,347	539,526	463,885	388,197	312,500
64B	U	195,261	190,399	175,774	175,774	175,782
	C	309,999	271,666	233,127	194,709	156,250
32B	U	117,140	112,278	97,653	97,653	97,657
	C	155,596	136,260	116,872	97,512	78,125
16B	U	78,079	73,217	58,592	58,592	58,594
	C	77,953	68,259	58,530	39,063	39,063
8B	U	58,549	53,687	39,062	39,062	39,063
	C	39,019	34,157	19,532	19,532	19,532

Table VII. Number of Reduced Page Writes Due to the Deduplication

Record Length	Sorting Method	$rs = 0$	$rs = 0.25$	$rs = 0.5$	$rs = 0.75$	$rs = 1$
4KB	<i>U</i>	43	4,905	19,530	19,530	19,530
	<i>C</i>	9,999,999	9,999,999	9,999,999	9,999,999	9,999,999
2KB	<i>U</i>	43	4,905	19,530	19,530	19,530
	<i>C</i>	2,500,738	3,124,836	3,750,432	4,375,323	4,999,999
1KB	<i>U</i>	43	4,905	19,530	19,530	19,530
	<i>C</i>	625,051	1,094,010	1,563,110	2,031,537	2,499,999
512B	<i>U</i>	43	4,905	19,530	19,530	19,530
	<i>C</i>	156,695	430,098	702,811	976,534	1,249,999
256B	<i>U</i>	43	4,905	19,530	19,530	19,530
	<i>C</i>	39,517	185,635	332,124	478,431	624,999
128B	<i>U</i>	43	4,905	19,530	19,530	19,530
	<i>C</i>	9,652	85,473	161,114	236,802	312,499
64B	<i>U</i>	43	4,905	19,530	19,530	19,530
	<i>C</i>	2,500	40,833	79,372	117,790	156,249
32B	<i>U</i>	43	4,905	19,530	19,530	19,530
	<i>C</i>	653	19,989	39,377	58,737	78,124
16B	<i>U</i>	43	4,905	19,530	19,530	19,530
	<i>C</i>	171	9,865	19,594	39,061	39,061
8B	<i>U</i>	43	4,905	19,530	19,530	19,530
	<i>C</i>	43	4,905	19,530	19,530	19,530

could not be reduced significantly and was even more than that of the clustered sorting. This was because the unclustered sorting could not gain benefits when the record length was small. For example, when the record length was 8B or 16B, the number of page writes of the unclustered sorting was more than that of the clustered sorting under any rs values. However, when the record length was increased or equal to a page size (e.g., 4KB), the clustered sorting could be favored by the concept of reused data and most writes could be reduced for the clustered sorting, as shown in Table VII. When rs was increased, the number of reduced page writes increased. This was because large record length and large rs could increase the probability that the content of the output buffer was identical to one page in flash memory. On the other hand, because the unclustered sorting is designed to sort tags instead of records, the number of reduced page writes of the unclustered sorting will not be affected by the record length.

5.1.3. Number of Block Erases. The number of block erases was proportional to the number of page writes. Because the number of page writes of the unclustered sorting in most cases was less than that of the clustered sorting, the number of block erases of the unclustered sorting was also less than that of the clustered sorting, as shown in Table VIII. Because writes can cause a sequence of garbage collection activities that contain page reads, page writes, and block erases, many writes and erases can damage the lifetime of flash memory. Therefore, we need to reduce the activities of writes and erases as much as possible. We observe that the unclustered sorting can reduce the number of page writes and block erases with larger record length. As a result, we can exploit the advantages of both clustered and unclustered sorting to reduce the garbage collection overhead.

5.1.4. Execution Time. We also calculated the execution time according to the number of page reads, page writes, and block erases with Table II. As shown in Table IX, the execution time of the unclustered sorting in some cases was faster than or close to that of the clustered sorting. When the record length was decreased to the tag size, the execution time of the unclustered sorting was inferior to that of the clustered sorting.

Table VIII. Number of Block Erases

Record Length	Sorting Method	$rs = 0$	$rs = 0.25$	$rs = 0.5$	$rs = 0.75$	$rs = 1$
4KB	U	156,851	156,775	156,547	156,547	156,547
	C	156,250	156,250	156,250	156,250	156,250
2KB	U	78,730	78,654	78,426	78,426	78,430
	C	117,175	107,424	97,649	87,885	78,125
1KB	U	39,670	39,594	39,365	39,365	39,367
	C	68,358	61,031	53,701	46,382	39,062
512B	U	20,139	20,063	19,835	19,835	19,836
	C	36,614	32,342	28,081	23,804	19,531
256B	U	10,374	10,298	10,070	10,070	10,070
	C	18,913	16,630	14,341	12,055	9,765
128B	U	5,492	5,416	5,187	5,187	5,188
	C	9,614	8,430	7,248	6,065	4,882
64B	U	3,050	2,974	2,746	2,746	2,746
	C	4,843	4,244	3,642	3,042	2,441
32B	U	1,830	1,754	1,525	1,525	1,525
	C	2,431	2,129	1,826	1,523	1,220
16B	U	1,219	1,144	915	915	915
	C	1,218	1,066	914	610	610
8B	U	914	838	610	610	610
	C	609	533	305	305	305

Table IX. Execution Time (seconds)

Record Length	Sorting Method	Improvement Ratio	$rs = 0$	$rs = 0.25$	$rs = 0.5$	$rs = 0.75$	$rs = 1$
		C is a baseline					
4KB	U	0.998	2,743.9	2,742.8	2,739.5	2,739.5	2,739.5
	C	1	2,734.4	2,734.4	2,734.4	2,734.4	2,734.4
2KB	U	1.154	1,486.3	1,456.1	1,424.2	1,396.2	1,372.5
	C	1	1,925.6	1,786.2	1,646.4	1,506.8	1,367.2
1KB	U	1.171	844.6	801.9	758	719.2	688.9
	C	1	1,102.5	997.7	893	788.3	683.6
512B	U	1.113	499.4	454.8	410.1	372.4	347.1
	C	1	586.1	525	464.1	402.9	341.8
256B	U	1.041	293.5	255.5	219	191.2	176.2
	C	1	301.7	269.1	236.3	203.6	170.9
128B	U	0.996	161	136.1	113	97.7	90.8
	C	1	153.1	136.2	119.3	102.4	85.4
64B	U	0.971	82.8	69.8	57.1	50.7	48.1
	C	1	77.1	68.5	59.9	51.3	42.7
32B	U	0.931	42.2	35.6	29.6	27.2	26.7
	C	1	38.7	34.4	30	25.7	21.4
16B	U	0.789	23.3	20.7	16.5	16	16
	C	1	19.4	17.2	15	10.7	10.7
8B	U	0.553	15.5	14.3	10.7	10.7	10.7
	C	1	9.7	8.6	5.3	5.3	5.3

This was because the unclustered sorting could not benefit from the smaller record length. Additional page writes were needed for the management of tags and the record rearrangement/replacement. On the other hand, when rs was increased, the unclustered and clustered sorting could have faster execution time. Obviously, being able to detect if the output buffer was identical to some page in flash memory could reduce a lot of page writes for both sorting methods, especially when rs was increased. Overall, the improvement ratio of the unclustered sorting to the clustered sorting was about 1.041 to 1.171, when the record length was between 256B and 2KB. However, when the record length was equal to a page size (e.g., 4KB), the unclustered sorting could not be favored because no buffering occurred during the record rearrangement/replacement.

5.2. Observations

Based on these experimental results, we can provide observations about both clustered and unclustered sorting for NAND flash-based storage systems. The advantage of the clustered sorting is that all sorted records can be in a contiguous space and disk-based storage systems can have better performance because of spatial locality (i.e., small mechanical latency). The disadvantage of the clustered sorting is that the merging process of runs will result in a large number of page reads/writes and block erases. In contrast with the clustered sorting, the advantage of the unclustered sorting is that small tags can reduce the number of page reads and page writes during the merging process of runs. The disadvantage of the unclustered sorting is that additional page reads can be produced during the record rearrangement. To follow, we will address two factors influencing the clustered and unclustered sorting. Then, we can propose a decision rule for both sorting methods by exploiting the advantages of each.

- Value distribution of primary keys:** When primary keys are randomly generated, a large number of page reads occur for the unclustered sorting during the record rearrangement. When primary keys are in an ascending (or descending) order, the large number of page writes for both sorting methods can be reduced significantly during the merging process of runs by the concept of reused data.
- Record length:** When the record length is large, the advantage of the unclustered sorting can be demonstrated because of the small tag size. For the fixed-length records, we can determine which sorting method should be adopted in advance based on the record length, because the unclustered sorting can benefit from large record length. For the variable-length records, different record lengths can have different benefits to both clustered and unclustered sorting. As a result, a decision rule of the clustered and unclustered sorting is needed.

5.3. A Decision Rule

We propose a decision rule of the clustered and unclustered sorting for variable-length records. Because the record length can play an important role, the decision rule must consider it. Assume that all variable-length records are in Φ . Let Ψ be a record set and $\forall rec \in \Psi$; rec 's length is $\geq P$ and is not a multiple of the page size. We list four steps to explain the proposed sorting method as follows:

- Step 1: Scan all variable-length records in Φ .
- Step 2: Based on the decision rule, we can separate Ψ from Φ .
- Step 3: We apply the unclustered sorting for records in Ψ and the clustered sorting for records in $\Phi - \Psi$.
- Step 4: Merge the sorted records in $\Phi - \Psi$ and Ψ .

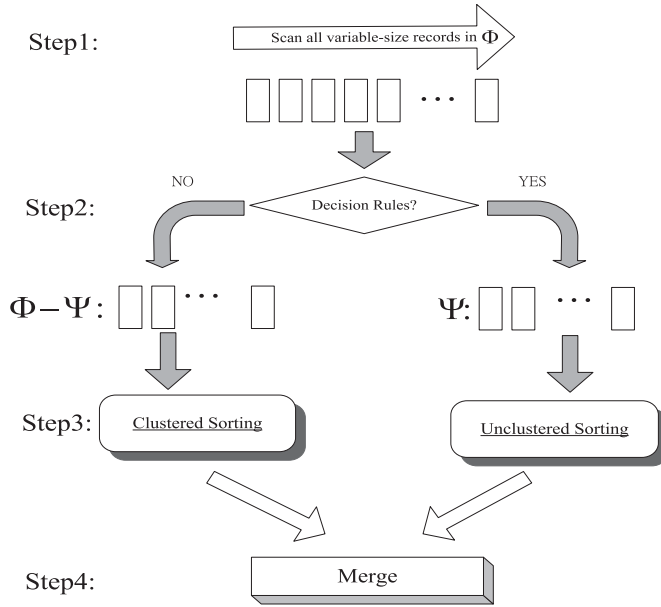


Fig. 5. A decision rule for the clustered and unclustered sorting.

We will define a decision rule to determine which sorting method should be applied to the variable-length records. Note that the decision rule can let the suitable sorting method be applied to records based on the observations and I/O analysis.

Decision rule. If the record length is $\geq P$ and is not a multiple of the page size, then we will apply the unclustered sorting to Ψ ; otherwise, we will apply the clustered sorting.

Because the unclustered sorting can benefit from large record length, P is just used to distinguish between large-length and small-length records. Although it is difficult to determine an optimal P for the decision rule under any situation (e.g., page size, record length, access patterns, and SSDs), we have set different values of P for the decision rule in the experiments, where P will vary from 8 to 512. Based on the experimental results, we can show that the decision rule can provide better and reasonable performance if P was reasonably selected (e.g., $P = 128$ or $P = 256$) and not extreme size. When the record length is a multiple of the page size, it is Case 2 in the record rearrangement and replacement of the unclustered sorting. Because Case 2 will not gain benefits from the buffer pages during the record rearrangement and replacement, Case 2 can degrade the performance of the unclustered sorting. Therefore, the decision rule should avoid Case 2 in the unclustered sorting.

As shown in Figure 5, the decision rule for the clustered and unclustered sorting is described by the four steps. Regardless of which sorting method is used, Step 1 is needed for generating runs in the clustered sorting or for generating tags in the unclustered sorting. In Step 2, we separate Ψ from Φ based on the decision rule. In Step 3, we apply the unclustered sorting to records in Ψ and the clustered sorting to records in $\Phi - \Psi$. In Step 4, we can merge the sorted records in $\Phi - \Psi$ and Ψ . As a result, the four steps can overlap with the original operations of both clustered and unclustered sorting. Therefore, the decision rule for both clustered and unclustered sorting will not

Table X. Parameter Definition

Number of pages of a large file	N
Number of buffer pages (i.e., main memory)	B
Size of a record	γ
Size of a page	p
Size of a tag	m
Number of pages in a block	np

Table XI. I/O Analysis of the Unclustered Sorting

I/O Cost	$Y = GC + SC + EC + RR$
GC	$N * t_r + (\lceil N * \frac{m}{\gamma} \rceil) * t_w$
SC	$(\lceil N * \frac{m}{\gamma} \rceil * \log_{B-1} \lceil \frac{N * m}{B * \gamma} \rceil) * (t_r + t_w)$
EC	$((\lceil N * \frac{m}{\gamma} \rceil + \lceil N * \frac{m}{\gamma} \rceil * \log_{B-1} \lceil \frac{N * m}{B * \gamma} \rceil) / np) * t_e$
RR	$N * t_w + \text{page reads during the record rearrangement and replacement}$

result in additional overhead. In the following section, we will give insights into the decision rule by I/O analysis.

6. I/O ANALYSIS

6.1. I/O Analysis of the Clustered and Unclustered Sorting

In this section, we will derive an I/O analysis of the clustered and unclustered sorting for NAND flash-based storage systems. Table X shows the parameter definition. t_r , t_w , and t_e denote the operation time for a page read, a page write, and a block erase, respectively. We distinguish between page reads and page writes because reads and writes are asymmetrical operations on flash memory. Reads on flash memory always have less overhead than writes on flash memory. Based on Table II, the access speed of a page read is about eight times faster than that of a page write. Page writes can incur a sequence of garbage collection activities that contain page reads, page writes, and block erases. Furthermore, a large number of pages writes and block erases can reduce the lifetime of flash memory. That is why we should distinguish between page reads and page writes when using flash memory. The I/O cost of the clustered sorting can be defined as follows: $N * (t_r + t_w)$ is to generate runs in the first phase by reading N pages and writing N pages. $(N * \log_{B-1} \lceil N/B \rceil) * (t_r + t_w)$ is the cost to merge the runs in the second phase, where $\lceil N/B \rceil$ is the number of runs, $(\log_{B-1} \lceil N/B \rceil)$ is the merge count, and each merge will require N page reads and N page writes. $((N + N * \log_{B-1} \lceil N/B \rceil) / np) * t_e$ is the average erase cost during the first phase and the second phase. Assume that writes in the clustered sorting are in an appending (and sequential) fashion; one block erase will be needed after np page writes. In the following equation, X denotes the I/O cost of the clustered sorting:

$$X = N * (t_r + t_w) + (N * \log_{B-1} \lceil N/B \rceil) * (t_r + t_w) + ((N + N * \log_{B-1} \lceil N/B \rceil) / np) * t_e. \quad (1)$$

We also summarize the I/O analysis for the unclustered sorting, as shown in Table XI. Let GC denote the generation cost of a tag file. GC will need N page reads and $\lceil N * \frac{m}{\gamma} \rceil$ page writes for the tag file. Let SC denote the sorting cost of the tag file. SC will need $(\log_{B-1} \lceil \frac{N * m}{B * \gamma} \rceil)$ merge count, and each merge will require $\lceil N * \frac{m}{\gamma} \rceil$ page reads and page writes. Let EC denote the recycling cost during sorting. EC will need $((\lceil N * \frac{m}{\gamma} \rceil + \lceil N * \frac{m}{\gamma} \rceil * \log_{B-1} \lceil \frac{N * m}{B * \gamma} \rceil) / np)$ block erases, because one block erase will be needed

Table XII. I/O Comparison of the Clustered Sorting, the Unclustered Sorting, MinSort, and FAST

	Number of Page Reads	Number of Page Writes
Clustered sorting	$N + (N * \log_{B-1} \lceil N/B \rceil)$	$N + (N * \log_{B-1} \lceil N/B \rceil)$
Unclustered sorting	$\overbrace{N + (\lceil N * \frac{m}{\gamma} \rceil * \log_{B-1} \lceil \frac{N * m}{B * \gamma} \rceil)}^{\text{The sorting cost of the tag file}} + \text{page reads during } RR$	$\overbrace{(\lceil N * \frac{m}{\gamma} \rceil) + (\lceil N * \frac{m}{\gamma} \rceil * \log_{B-1} \lceil \frac{N * m}{B * \gamma} \rceil) + N}^{\text{The sorting cost of the tag file}}$
MinSort	$N * (1 + D_r)$	N
FAST	$\lceil N/Q \rceil * \lceil Q/B \rceil * Q + \sum_{h=2}^{\lceil \log_Q N \rceil} (\lceil N/Q^h \rceil * (\lceil Q^h/B \rceil * Q + Q^h))$	$2 * N$

after np page writes. Let RR denote the I/O cost during the record rearrangement and replacement. Let Y denote the I/O cost of the unclustered sorting. Assume that $X > Y$ and then $X > GC + SC + EC + RR$. We can derive the following inequality: Let $\frac{m}{\gamma}$ be α , where $0 < \alpha \leq 1$:

$$\begin{aligned}
& X - GC - SC - EC > RR \\
\Rightarrow & (\lceil (1 - \alpha) * N \rceil) * t_w + (\lceil (1 - \alpha) * N \rceil \\
& \quad * \log_{B-1} \lceil N/B \rceil - \lceil \alpha * N \rceil * \log_{B-1} \alpha) * (t_r + t_w) \\
& \quad + ((\lceil (1 - \alpha) * N \rceil + \lceil (1 - \alpha) * N \rceil * \log_{B-1} \lceil N/B \rceil \\
& \quad - \lceil \alpha * N \rceil * \log_{B-1} \alpha) / np) * t_e > RR \\
\Rightarrow & \underbrace{N * t_w + (N * \log_{B-1} \lceil N/B \rceil) * (t_r + t_w)}_{\text{Let } \alpha \text{ be 0 and the left side will become the largest}} \\
& \quad + ((N + N * \log_{B-1} \lceil N/B \rceil) / np) * t_e > RR. \tag{2}
\end{aligned}$$

According to Inequality (2), we can make the following observations:

- If $X > Y$, RR must be as small as possible. In other words, I/O cost during the record rearrangement and replacement should be reduced as much as possible. RR could be influenced significantly by the value distribution of primary keys due to the record rearrangement and replacement. As a result, we propose the concept of reused data and the buffering capability to decrease the number of page reads.
- When γ (i.e., the record length) is increased, α is decreased. If α is close to 0, the left side in Inequality (2) will be the largest. In other words, a larger record length can reduce the number of page writes and block erases because GC , SC , and EC can be decreased accordingly. A larger record length can also make the left side in Inequality (2) larger and RR can compensate for a large number of page reads during the record rearrangement and replacement.

6.2. I/O Comparison of the Clustered Sorting, the Unclustered Sorting, MinSort, and FAST

In Table XII, we show the I/O comparison of the clustered sorting, the unclustered sorting, MinSort, and FAST in terms of number of page reads and writes. In particular, D_r is the average number of distinct values stored per page in MinSort, and Q is the region size in FAST. As mentioned earlier, the advantage of the clustered sorting is that all sorted records can be in a contiguous space and disk-based storage systems can have better performance because of spatial locality (i.e., small mechanical latency), and the advantage of the unclustered sorting is that small tags can reduce the number of page reads and page writes during the merging process of runs, especially when the record length is large. We can observe that the number of page writes of the unclustered

sorting consists of the sorting cost of the tag file and N page writes. That is, if the ratio of m to γ is less than a threshold, $((\lceil N * \frac{m}{\gamma} \rceil) + (\lceil N * \frac{m}{\gamma} \rceil * \log_{B-1} \lceil \frac{N * m}{B * \gamma} \rceil) + N)$ can be less than a value (e.g., $2 * N$ or $N + (N * \log_{B-1} \lceil N/B \rceil)$). Therefore, the number of page writes of the unclustered sorting can be reduced and less than that of FAST or the clustered sorting when the record length (i.e., γ) is large. Although the number of page writes of MinSort is N page writes, its number of page reads will be affected by D_r . The number of page reads of MinSort (i.e., $N * (1 + D_r)$) will increase significantly when the number of distinct values stored per page is large (i.e., random distribution of primary keys). For the number of page reads of FAST, it can be affected by Q . According to the derivation, the number of page reads of FAST is about $O(N + B * N^2)$ and $O(\frac{8 * N^2}{B})$, if Q is set at B and 2 (i.e., $2 \leq Q \leq B$), respectively. If B is much smaller than N , the number of page reads of FAST is very large. On the other hand, because the record rearrangement and replacement (i.e., RR) can use the sorted tags to replace records that will not be used for the longest period of time, the number of page reads of the unclustered sorting can be reduced significantly and less than that of MinSort or FAST. Therefore, the unclustered sorting can provide better performance than the clustered sorting, MinSort, and FAST by considering the number of page reads and writes. In addition, based on the I/O analysis and observations, the unclustered sorting is suitable for large record length, and the clustered sorting is suitable for small record length. As a result, the decision rule can exploit the advantages of both clustered and unclustered sorting to determine which sorting method should be applied to the variable-length records.

7. PERFORMANCE EVALUATION

7.1. Experimental Setup and Performance Metrics

The experimental environment was built using a desktop with an Intel Core i7 CPU at 3.07GHz with a 12GB DRAM. In the implementation, an Intel 710 SSD (SSDSA2BZ100G3) with 100GB capacity and a Hitachi HDD (HDP725050GLA360) with 500GB capacity were used as the storage system. An 8B tag was used for the unclustered sorting. Because we conducted experiments in real storage devices including a real SSD (i.e., an Intel SSD) and a hard disk drive (i.e., a Hitachi HDD), we can measure the actual I/O workloads to the storage devices in terms of “number of 4KB reads” and “number of 4KB writes,” where 4KB is an access unit in the file system (i.e., NTFS). We have set different values of P for the decision rule to compare the clustered sorting, the unclustered sorting, MinSort [Cossentine and Lawrence 2013], and FAST [Park and Shim 2009]. For example, $D < P = 128 >$ means that the decision rule is used with $P = 128$. P will vary from 8 to 512 in the experiments (i.e., $D < P = 8 >$, $D < P = 16 >$, $D < P = 32 >$, $D < P = 64 >$, $D < P = 128 >$, $D < P = 256 >$, and $D < P = 512 >$). We also distinguish the execution ratio between the clustered sorting and the unclustered sorting in the decision rule. Note that Table XIII shows different configurations for the decision rule, the clustered sorting, the unclustered sorting, MinSort, and FAST. In particular, $D < P = 8 >$ and the unclustered sorting are equivalent according to the definition of the decision rule.

In Section 7.2, we used a large dataset (i.e., about 40GB) to measure the I/O cost of the decision rule, the clustered sorting, the unclustered sorting, MinSort, and FAST by measuring the number of 4KB reads, the number of 4KB writes, the number of increased 4KB reads, the number of reduced 4KB writes, the normalized execution time, and the standard deviation. In Section 7.3, we generated two real datasets to measure the I/O cost of the decision rule, the clustered sorting, the unclustered sorting, MinSort, and FAST by measuring the number of 4KB reads, the number of 4KB writes, the number of increased 4KB reads, the number of reduced 4KB writes, the normalized execution time, and the standard deviation. Each experiment was performed five times

Table XIII. Configurations

D<P=8>-HDD	Decision rule with $P = 8$ for a Hitachi HDD
D<P=8>-SSD	Decision rule with $P = 8$ for an Intel SSD
D<P=16>-HDD	Decision rule with $P = 16$ for a Hitachi HDD
D<P=16>-SSD	Decision rule with $P = 16$ for an Intel SSD
D<P=32>-HDD	Decision rule with $P = 32$ for a Hitachi HDD
D<P=32>-SSD	Decision rule with $P = 32$ for an Intel SSD
D<P=64>-HDD	Decision rule with $P = 64$ for a Hitachi HDD
D<P=64>-SSD	Decision rule with $P = 64$ for an Intel SSD
D<P=128>-HDD	Decision rule with $P = 128$ for a Hitachi HDD
D<P=128>-SSD	Decision rule with $P = 128$ for an Intel SSD
D<P=256>-HDD	Decision rule with $P = 256$ for a Hitachi HDD
D<P=256>-SSD	Decision rule with $P = 256$ for an Intel SSD
D<P=512>-HDD	Decision rule with $P = 512$ for a Hitachi HDD
D<P=512>-SSD	Decision rule with $P = 512$ for an Intel SSD
C-HDD	Clustered sorting C for a Hitachi HDD
C-SSD	Clustered sorting C for an Intel SSD
U-HDD	Unclustered sorting U for a Hitachi HDD
U-SSD	Unclustered sorting U for an Intel SSD
MinSort-HDD	MinSort sorting for a Hitachi HDD
MinSort-SSD	MinSort sorting for an Intel SSD
FAST-HDD	FAST sorting for a Hitachi HDD
FAST-SSD	FAST sorting for an Intel SSD

to obtain an average. In particular, D<P=128>-SSD is the comparison baseline for the normalized execution time. In order to set the better FAST configuration, its region size (i.e., Q) is set to the buffer size (i.e., B) because FAST consists of two phases (i.e., run generation and run merging). In order to set the better MinSort configuration, the ability of record rearrangement and replacement can be added into MinSort to decrease the number of page reads. However, an index array in MinSort for maintaining the minimum values could consume the buffer pages. According to the experimental results, FAST could behave like the clustered sorting when its region size (i.e., Q) is equal to the buffer size (i.e., B), and MinSort could behave like the unclustered sorting due to the ability of record rearrangement and replacement.

7.2. Large Dataset

In the experiments, we adopted 100,000,000 records in a 40GB record file and the size of available main memory was set at 200MB. The capacity size of the Intel 710 SSD is 100GB initially and becomes about 93GB after formatting with NTFS in Windows 7. Therefore, the 40GB size is appropriate in case the intermediate data (e.g., merging runs) and the dataset coexist in the Intel 710 SSD. The record length was randomly chosen from the set of (8B, 16B, 32B, 64B, 512B, 1KB, 2KB, and 4KB), and the primary keys of all records were randomly generated.

We can observe that the number of 4KB reads of the unclustered sorting was more than that of the clustered sorting and FAST, as shown in Figure 6(a). This was because randomly selected primary keys might cause overhead for record rearrangement and replacement, and the number of 4KB reads increased. The number of 4KB reads of the decision rule (e.g., D<P=8>, D<P=16>, D<P=32>, D<P=64>, D<P=128>, D<P=256>, and D<P=512>) was less than that of the unclustered sorting and MinSort, but more than that of the clustered sorting and FAST. Because the decision rule can utilize the advantages of both clustered and unclustered sorting, the decision rule can decrease 4KB reads during the merging process of runs. Although MinSort added

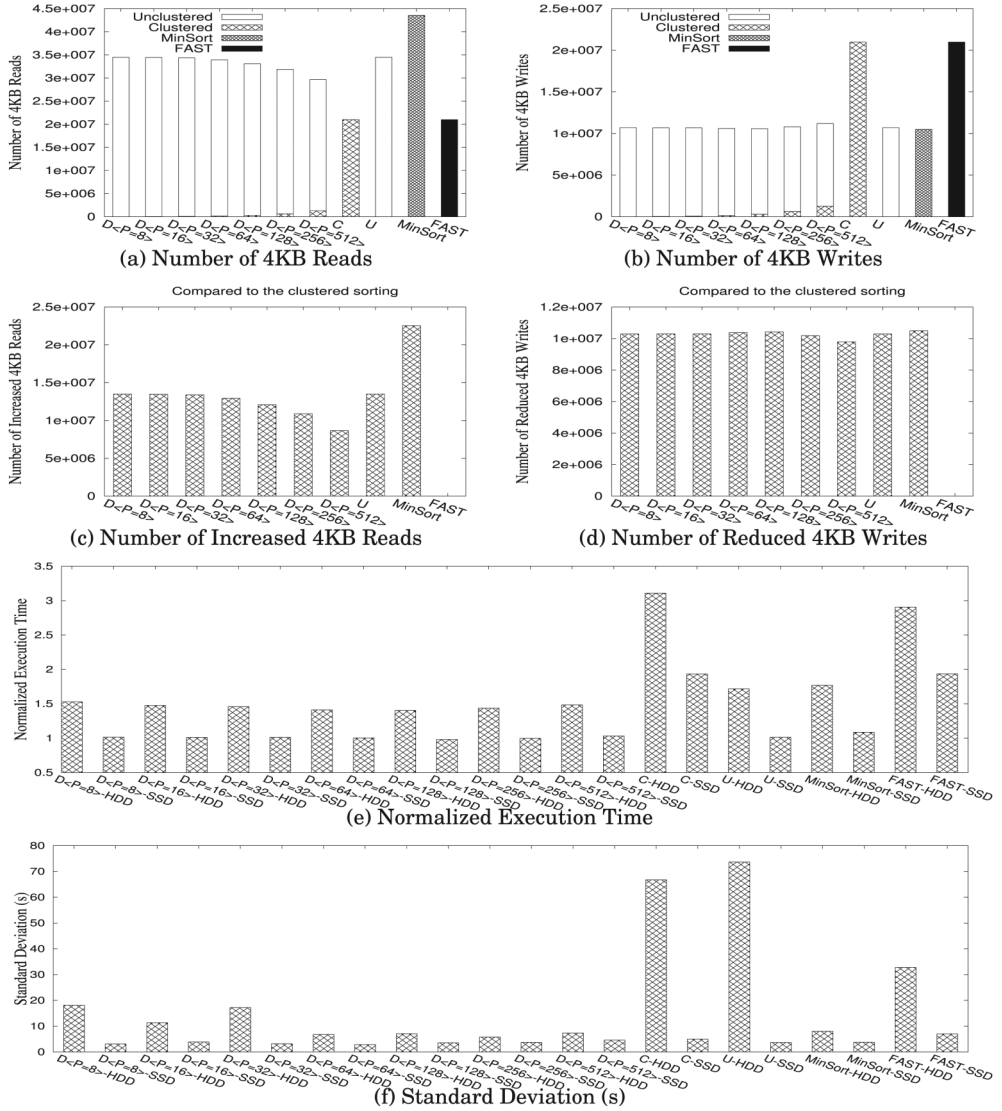


Fig. 6. I/O cost of a large dataset.

the ability of record rearrangement and replacement, its index array for maintaining the minimum values could consume the buffer pages and caused more 4KB reads than the unclustered sorting. Also, when the record length was variable, the decision rule could adopt the clustered sorting for small record length and the unclustered sorting for large record length. This was because the unclustered sorting could reduce 4KB writes by using small tags. The number of 4KB writes of the decision rule was a little more than that of the unclustered sorting and MinSort, as shown in Figure 6(b). This was because some 4KB writes have been generated during the merging process in Step 4 of the decision rule. However, the number of 4KB reads of the decision rule was less than that of the clustered sorting and FAST. This was because the clustered sorting

and FAST must store sorted runs in the storage system at both the generation and merge phases of runs and caused a large number of 4KB writes.

Compared to the clustered sorting, we measured the number of increased 4KB reads and the number of reduced 4KB writes, as shown in Figures 6(c) and 6(d). We can observe that the number of increased 4KB reads for the decision rule was less than that of the unclustered sorting and MinSort. Moreover, the number of reduced 4KB writes for the decision rule, the unclustered sorting, and MinSort was larger than FAST. Therefore, the decision rule can reduce 4KB writes significantly and increase 4KB reads slightly. When an SSD was used, we can observe that the normalized execution time of the decision rule (e.g., $D<P=8>$ -SSD, $D<P=16>$ -SSD, $D<P=32>$ -SSD, $D<P=64>$ -SSD, $D<P=128>$ -SSD, $D<P=256>$ -SSD, and $D<P=512>$ -SSD) was better than that of the clustered sorting, the unclustered sorting, MinSort, and FAST, as shown in Figure 6(e). This was because the decision rule could exploit the advantages of both clustered and unclustered sorting. When an HDD was used, we can observe that the normalized execution time of all methods was worse than that with an SSD. This was because many random primary keys could let the HDD generate a lot of seek operations, which might cause a lot of overhead, especially for C-HDD, MinSort-HDD, and FAST-HDD. We also measured the standard deviation of the execution time for all methods under an SSD and an HDD, as shown in Figure 6(f). It is obvious that the HDD caused a larger standard deviation than that of the SSD. This was because the SSD had no mechanical latency and could provide more stable performance than the HDD.

Overall, U-SSD, $D<P=8>$ -SSD, $D<P=16>$ -SSD, $D<P=32>$ -SSD, $D<P=64>$ -SSD, $D<P=128>$ -SSD, $D<P=256>$ -SSD, and $D<P=512>$ -SSD can have better normalized execution time. This was because a lot of seek operations didn't have a significant impact on SSDs. Also, because the unclustered sorting and the decision rule could reduce 4KB reads during record rearrangement and replacement, seek operations could be reduced. We can observe that when P was increased, the execution ratio of the clustered sorting in the decision rule increased, as shown in Figures 6(a) and 6(b). Although it is difficult to determine an optimal P for the decision rule, we can demonstrate the efficiency and practicality of the decision rule with reasonable values of P . According to the experimental results, the decision rule could provide better and reasonable performance if P was about 128 (i.e., $D<P=128>$ -SSD) when page size was 4KB. As a result, the decision rule can provide an efficient external sorting for NAND flash-based storage systems if P is reasonably selected and not extreme size.

7.3. Real Dataset

In the experiments, we used two real datasets to measure the I/O cost of the decision rule, the clustered sorting, the unclustered sorting, MinSort, and FAST. The available main memory size was 2MB.

The first real dataset is "Fpers" [Fpers 2015], which is a personal contact list. The Fpers dataset has 10 attribute columns such as user name, phone number, and email address. Each record size was 128B, and user name could be used as a primary key. We generated 128,273 records from Fpers and the primary keys of all records were randomly generated. Because the record size was 128B, $D<P=8>$, $D<P=16>$, $D<P=32>$, $D<P=64>$, $D<P=128>$, and the unclustered sorting (i.e., U) are equivalent. $D<P=256>$, $D<P=512>$, and the clustered sorting (i.e., C) are also equivalent according to the definition of the decision rule.

We can observe that the number of 4KB reads of the unclustered sorting (i.e., U) and the decision rule (i.e., $D<P=8>$, $D<P=16>$, $D<P=32>$, $D<P=64>$, and $D<P=128>$) was a little more than that of the clustered sorting (i.e., C) and FAST, as shown in Figure 7(a). Although MinSort added the ability of record rearrangement and replacement,

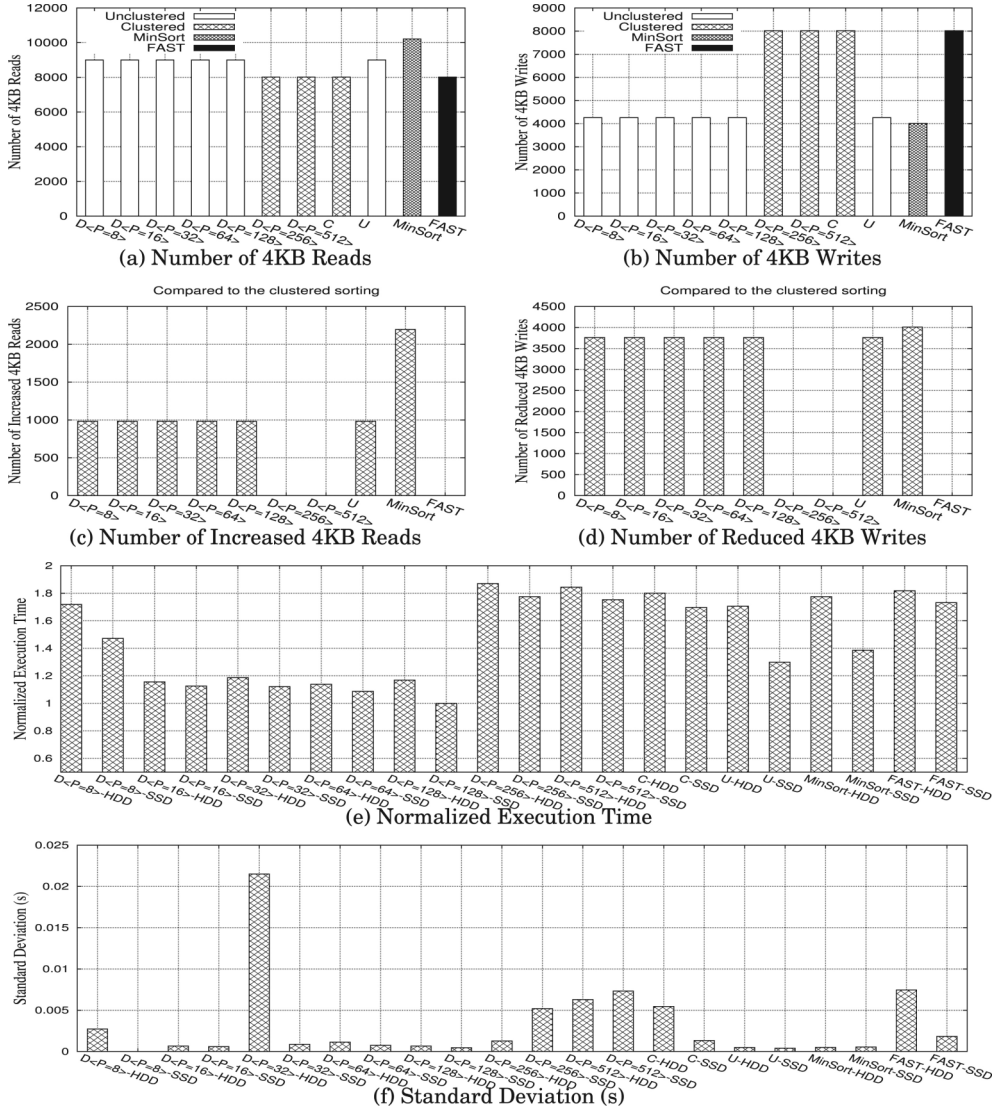


Fig. 7. I/O cost of real datasets: Fpers.

its index array for maintaining the minimum values could consume the buffer pages and caused more 4KB reads than the unclustered sorting. In addition, the number of 4KB writes of the unclustered sorting (i.e., U) and the decision rule (i.e., $D<P=8>$, $D<P=16>$, $D<P=32>$, $D<P=64>$, and $D<P=128>$) was less than that of the clustered sorting and FAST, as shown in Figure 7(b). The number of 4KB writes of MinSort was similar to the unclustered sorting and the decision rule because MinSort only wrote the sorted data once. However, FAST wrote two times the sorted data according to its method. Compared to the clustered sorting, we measured the number of increased 4KB reads and the number of reduced 4KB writes, as shown in Figures 7(c) and 7(d). We can observe that the number of increased 4KB reads for the decision rule (i.e., $D<P=8>$, $D<P=16>$, $D<P=32>$, $D<P=64>$, and $D<P=128>$) and the unclustered sorting (i.e.,

U) was less than that of MinSort. Moreover, the number of reduced 4KB writes for the decision rule (i.e., $D<P=8>$, $D<P=16>$, $D<P=32>$, $D<P=64>$, and $D<P=128>$), the unclustered sorting (i.e., U), and MinSort was larger than FAST. Therefore, the decision rule can reduce 4KB writes significantly and increase 4KB reads slightly. Note that because $D<P=256>$, $D<P=512>$, and the clustered sorting (i.e., C) are equivalent in this case, the number of increased 4KB reads and the number of reduced 4KB writes were zero.

We measured the normalized execution time, as shown in Figure 7(e). When an SSD was used, the normalized execution time of the decision rule (e.g., $D<P=8>$ -SSD, $D<P=16>$ -SSD, $D<P=32>$ -SSD, $D<P=64>$ -SSD, and $D<P=128>$ -SSD) was better than that of the clustered sorting, the unclustered sorting, MinSort, and FAST. Because seek operations didn't have a significant impact on SSDs, the unclustered sorting and the decision rule can utilize the advantages of SSDs. Also, because the unclustered sorting and the decision rule could reduce 4KB reads during record rearrangement and replacement, the unclustered sorting and the decision rule could be better than the clustered sorting. When an HDD was used, the normalized execution times of all methods are a little worse than that with an SSD. For example, C-HDD had a longer execution time than C-SSD because the random primary keys could cause a lot of seek operations for the clustered sorting and the unclustered sorting. We also measured the standard deviation of the execution time for all methods under an SSD and an HDD, as shown in Figure 7(f). It is obvious that the HDD caused larger standard deviation than that of the SSD. This was because the SSD has no mechanical latency and can provide more stable performance than the HDD.

The second real dataset is "Track Metadata" [2015]. The Track Metadata dataset is a freely available collection of audio features and metadata for a million contemporary popular music tracks, which has 14 attribute columns such as artist name, track id, release year, and artist hotness that can be used as a primary key. We generated 1,000,000 records from Track Metadata and the primary keys of all records were randomly generated. We randomly added additional information to the records such that each record length was among the set of (8B, 16B, 32B, 64B, 512B, 1KB, 2KB, and 4KB). The experimental results were similar to Fpers, as shown in Figure 8.

When an SSD was used, the normalized execution time of the decision rule (e.g., $D<P=8>$ -SSD, $D<P=16>$ -SSD, $D<P=32>$ -SSD, $D<P=64>$ -SSD, $D<P=128>$ -SSD, $D<P=256>$ -SSD, and $D<P=512>$ -SSD) was better than the clustered sorting, the unclustered sorting, MinSort, and FAST, as shown in Figure 8(e). Because seek operations didn't have a significant impact on SSDs, the decision rule can utilize the advantages of SSDs. When an HDD was used, the normalized execution time of all methods was worse than that with an SSD. This was because the random primary keys could let the HDD generate a lot of seek operations that might cause a lot of overhead, especially for C-HDD, MinSort-HDD, and FAST-HDD. We also measured the standard deviation of the execution time for all methods under an SSD and an HDD, as shown in Figure 8(f). It is obvious that the HDD caused larger standard deviation than that of the SSD. This was because the SSD had no mechanical latency and could provide more stable performance than the HDD.

8. CONCLUSION

There are four critical issues that significantly influence the efficiency of the external sorting in flash memory: (1) write once with bulk erase, (2) the endurance issue, (3) asymmetrical operations, and (4) no mechanical latency. With the very distinctive characteristics of flash memory, the typical external sorting method that adopts the clustered sorting could result in performance degradation of NAND flash-based storage systems and reduce the lifetime of flash memory. As a result, we adopt an unclustered

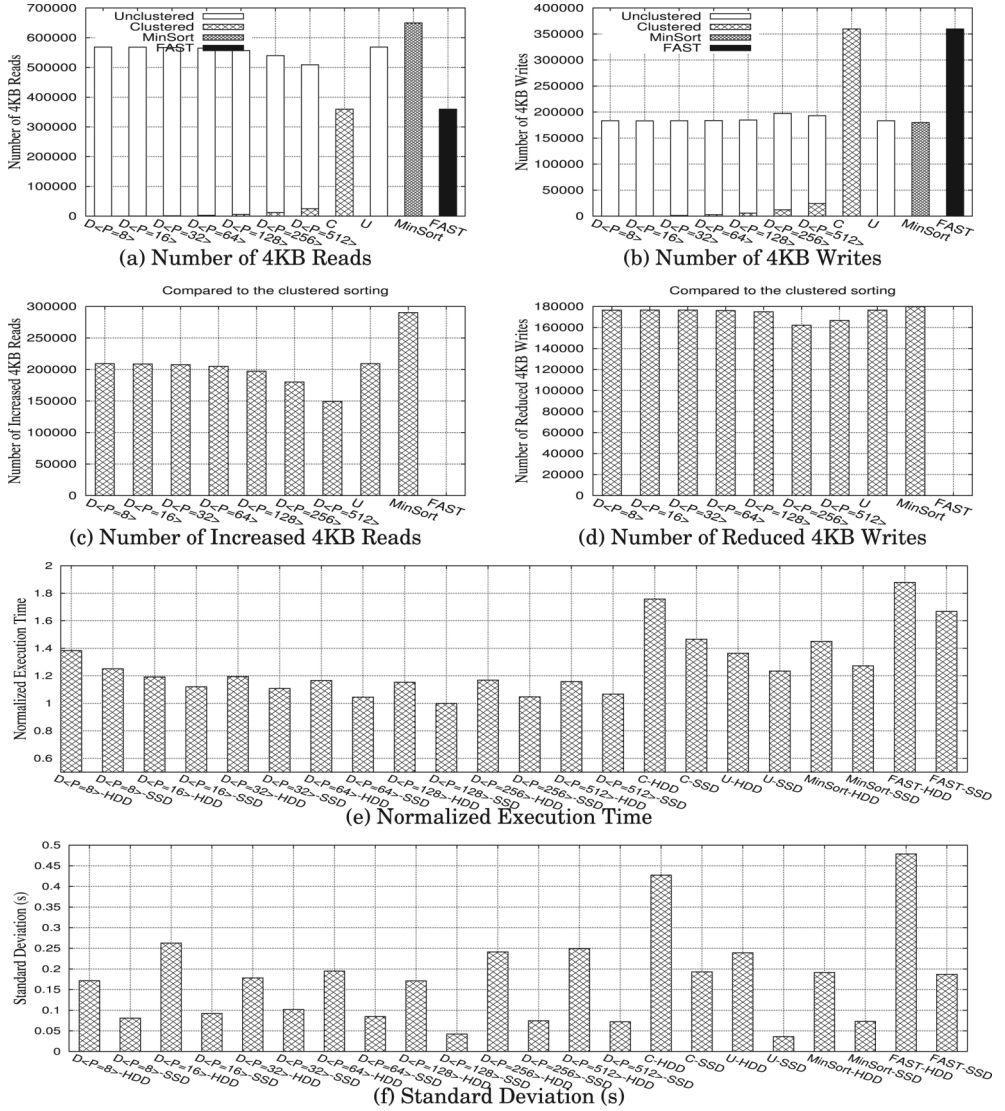


Fig. 8. I/O cost of real datasets: Track Metadata.

sorting method that considers the characteristics of flash memory. Because there are pros and cons for both clustered and unclustered sorting, a decision rule is proposed to exploit the advantages of both clustered and unclustered sorting. That is, the decision rule can separate records according to their record length, sort them appropriately by the clustered and unclustered sorting, and merge the sorted results. We also derive an I/O analysis of both clustered and unclustered sorting. We can explain why the value distribution of primary keys and the record length influence the external sorting by the I/O analysis. Moreover, we show the I/O comparison of the clustered sorting, the unclustered sorting, and the previous works (e.g., MinSort and FAST) in terms of number of page reads and writes. In the experiments, we have implemented the decision rule and compared the clustered sorting, the unclustered sorting, MinSort, and FAST

in a real SSD (i.e., an Intel 710 SSD) and a hard disk drive (i.e., a Hitachi HDD). We observe that the unclustered sorting is suitable for large record length, and the clustered sorting is suitable for small record length when the SSD is used. Therefore, the decision rule is based on the observation to combine the advantages of both clustered and unclustered sorting. According to the experimental results, we can demonstrate the efficiency and practicality of the decision rule. As a result, the decision rule can provide an efficient external sorting for NAND flash-based storage systems.

Because the record length plays an important condition in this article, the decision rule adopts the clustered and unclustered sorting to investigate the impact of the record length. In fact, the decision rule can be designed in a general and adaptive way to adopt other sorting methods (e.g., FAST and MinSort) to determine which appropriate sorting methods should be applied. Therefore, for future research, the decision rule can select appropriate sorting methods based on the most favorable conditions. In addition to the record length, the conditions can include database applications, hybrid storage systems, architectures (e.g., multicore CPUs or GPUs), and so forth.

REFERENCES

- Simona Boboila and Peter Desnoyers. 2011. Performance models of flash-based solid-state drives for real workloads. In *Proceedings of the IEEE Symposium on Mass Storage Systems and Technologies (MSST)*.
- Tyler Cossentine and Ramon Lawrence. 2013. Efficient external sorting on flash memory embedded devices. *International Journal of Database Management Systems (IJDMs)* 5, 1 (2013), Article No. 1.
- Peter Desnoyers. 2014. Analytic models of SSD write performance. *ACM Transactions on Storage (TOS)* 10, 2 (2014), Article No. 8.
- Samsung Electronics. 2011. *NAND Flash-Memory Datasheet and SmartMedia Data Book*.
- Fpers. 2015. Retrieved from <http://www.myflorida.com/dmsweb/411/fpers.dat>.
- Goetz Graefe. 2006. Implementing sorting in database systems. *Computer Surveys* 38, 3 (2006), Article No. 10.
- Donald E. Knuth. 1998. *Art of Computer Programming*. Addison-Wesley.
- Per-Ake Larson. 2003. External sorting: Run formation revisited. *IEEE Transactions on Knowledge and Data Eng.* 15, 4 (2003), 961–972.
- Per-Åke Larson and Goetz Graefe. 1998. Memory management during run generation in external sorting. *ACM SIGMOD Record* 27, 2 (1998), 472–483.
- Sang-Won Lee, Dong-Joo Park, Tae-Sun Chung, Dong-Ho Lee, Sangwon Park, and Ha-Joo Song. 2007. A log buffer-based flash translation layer using fully-associative sector translation. *ACM Transactions on Embedded Computing Systems* 6, 3 (2007), Article No. 18.
- Xiqing Li, Gordon Linoff, Stephen J. Smith, Craig Stanfill, and Kurt Thearling. 1993. A practical external sort for dhared disk MPPs. In *Proceedings of the ACM/IEEE Conference on Supercomputing*. 666–675.
- Track Metadata. 2015. Million Song Dataset. Retrieved from <http://labrosa.ee.columbia.edu/millionsong/pages/getting-dataset>.
- Hyoungmin Park and Kyuseok Shim. 2009. FAST: Flash-aware external sorting for mobile database systems. *Journal of Systems and Software* 82, 8 (2009), 1298–1312.
- Raghu Ramakrishnan and Johannes Gehrke. 2003. *Database Management Systems*. McGraw-Hill.
- Betty Salzberg, Alex Tsukerman, Jim Gray, Michael Stuewart, Susan Uren, and Bonnie Vaughan. 1990. A distributed single-input single-output external sort. In *Proceedings of the ACM SIGMOD Conference*. 94–101.
- Chin-Hsien Wu. 2012. Data sorting in flash memory. In *Proceedings of the ACM Symposium on Applied Computing (SAC)*. 1847–1849.
- John Yiannis and Justin Zobel. 2007. Compression techniques for fast external sorting. *International Journal on Very Large Data Bases* 16, 2 (2007), 269–291.
- Weiye Zhang and Per-Ake Larson. 1997. Dynamic memory adjustment for external mergesort. In *Proceedings of the 23rd International Conference on Very Large Data Bases (VLDB)*. 376–385.

Received August 2012; revised June 2014; accepted August 2014